AGENTIC CODING CLAUDE CODE VI



Build Production-Grade Enterprise Apps

AGENDA

Add Pre-Deploy Verifications Policy

Add the policy to CLAUDE.md.

Install Verification Tools

Install tools for formatting, linting, and type-checking for verification.

Install a Target Cloud Platform

Install and configure Fly.io, a cloud platform, as the deployment target.

Configure CI/CD Pipeline

Create the fly-deploy.yml YAML file as a GitHub Actions CI/CD configuration file that provides automated, continuous deployment (CD) for your app.

Test Pre-Deploy Verifications

Run bun run verify to ensure that all pre-deploy verifications pass.

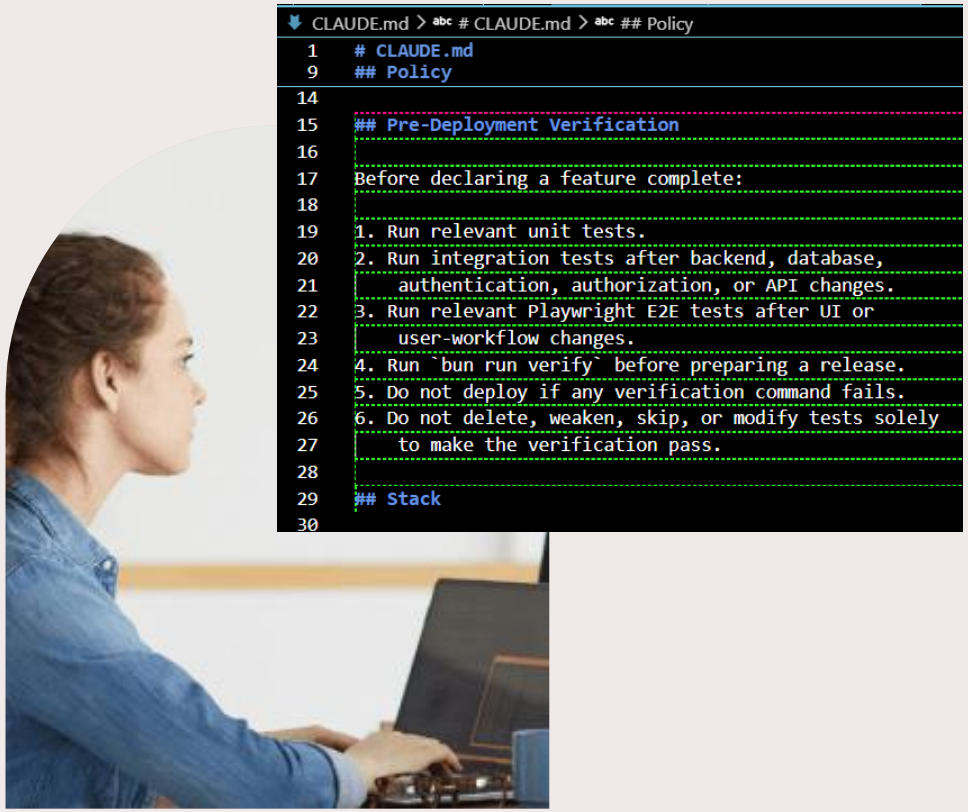
Run CI/CD Pipeline

Add "deploy": "bun run verify && flyctl deploy" script to package.json. Run bun run deploy, which invokes Fly's CLI tooling via flyctl to carry out deployment.

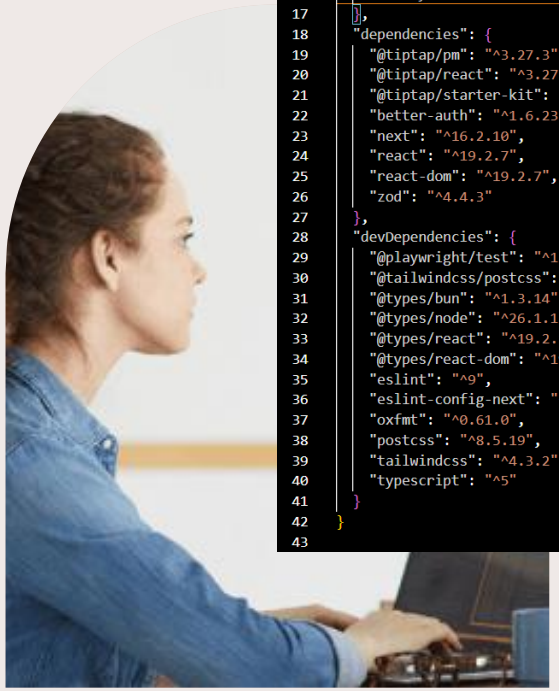
Quick Reference

Review of commands, paths, and keyboard combinations discussed in this session.

ADD PRE-DEPLOY VERIFICATIONS POLICY



- We will set up a **GitHub Actions** CI/CD workflow that runs all the verifications as a required gate before deployment.
- First, add the markdown shown on the left to **CLAUDE.md**.

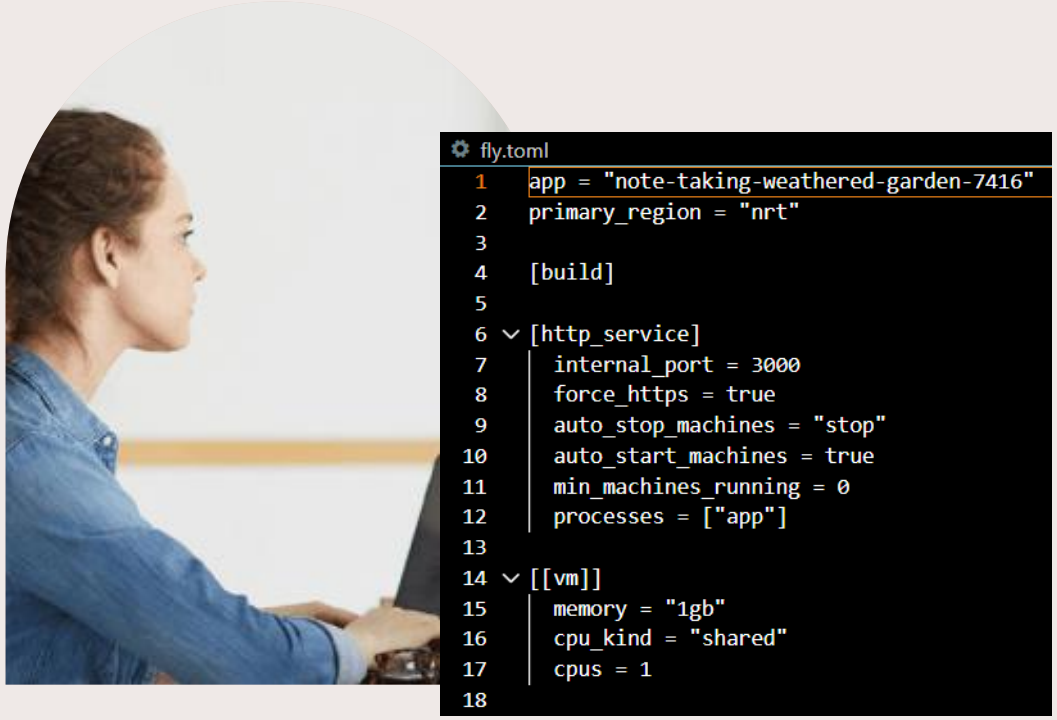


```
{} package.json > { } scripts > abc verify
{
    Debug
    "scripts": {
        "dev": "bun --bun next dev",
        "build": "bun --bun next build",
        "start": "bun --bun next start",
        "format": "oxfmt",
        "format:check": "oxfmt --check",
        "lint": "bunx eslint .",
        "typecheck": "bunx tsc --noEmit",
        "db:migrate": "bunx --bun auth@1.6.23 migrate --yes",
        "db:generate": "bunx --bun auth@1.6.23 generate --yes",
        "test:unit": "bun test tests/unit",
        "test:integration": "bun test tests/integration",
        "test:e2e": "playwright test",
        "test:all": "bun run test:unit && bun run test:integration && bun run test:e2e",
        "verify": "bun run format:check && bun run lint && bun run typecheck && bun run test:all"
    },
    "dependencies": {
        "@tiptap/pm": "^3.27.3",
        "@tiptap/react": "^3.27.3",
        "@tiptap/starter-kit": "^3.27.3",
        "better-auth": "^1.6.23",
        "next": "^16.2.10",
        "react": "^19.2.7",
        "react-dom": "^19.2.7",
        "zod": "^4.4.3"
    },
    "devDependencies": {
        "@playwright/test": "^1.61.1",
        "@tailwindcss/postcss": "^4.3.2",
        "@types/bun": "^1.3.14",
        "@types/node": "^26.1.1",
        "@types/react": "^19.2.17",
        "@types/react-dom": "^19.2.3",
        "eslint": "^9",
        "eslint-config-next": "^16.2.12",
        "oxfmt": "^0.61.0",
        "postcss": "^8.5.19",
        "tailwindcss": "^4.3.2",
        "typescript": "^5"
    }
}
```

INSTALL VERIFICATION TOOLS

- **Set up ESLint.**
  - **Run bun add -d eslint eslint-config-next to install ESLint tooling as dev dependencies in package.json.**
  - **Add ESLint Flat Config at eslint.config.mjs.**
  - **Run bun run lint to verify lint runs successfully.**
- **Define all required verification commands explicitly in package.json as shown on the left.**

INSTALL A TARGET CLOUD PLATFORM



fly.toml

```
app = "note-taking-weathered-garden-7416"
primary_region = "nrt"

[build]

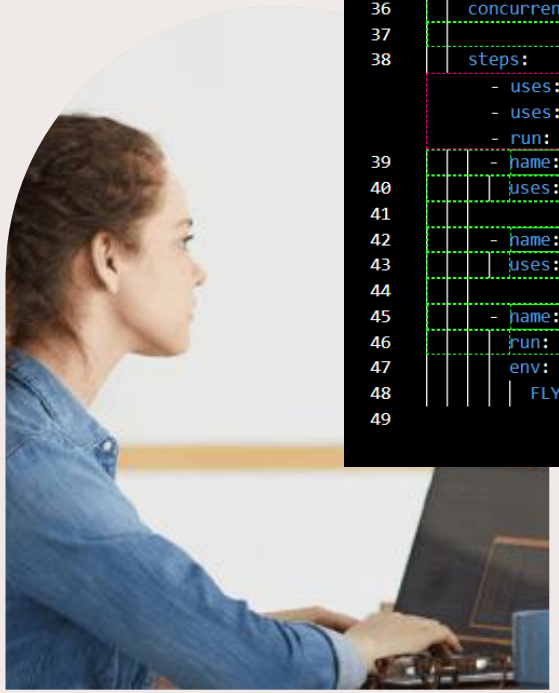
[http_service]
  internal_port = 3000
  force_https = true
  auto_stop_machines = "stop"
  auto_start_machines = true
  min_machines_running = 0
  processes = ["app"]

[[vm]]
  memory = "1gb"
  cpu_kind = "shared"
  cpus = 1

```

- Install **Fly.io** (a cloud platform used to deploy and run full-stack applications and containers globally) as the deploy target platform.
- Create the **fly.toml** file, which is the standard infrastructure configuration file for **Fly.io**.

CONFIGURE CI/CD PIPELINE



.github > workflows > fly-deploy.yml

```yaml
jobs:
  verify:
    steps:
      - name: Run verification suite
        run: bun run verify

  deploy:
    name: Deploy app
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    needs: verify
    runs-on: ubuntu-latest
    concurrency: deploy-group    # optional: ensure only one action runs at a time
    concurrency: fly-production-deploy

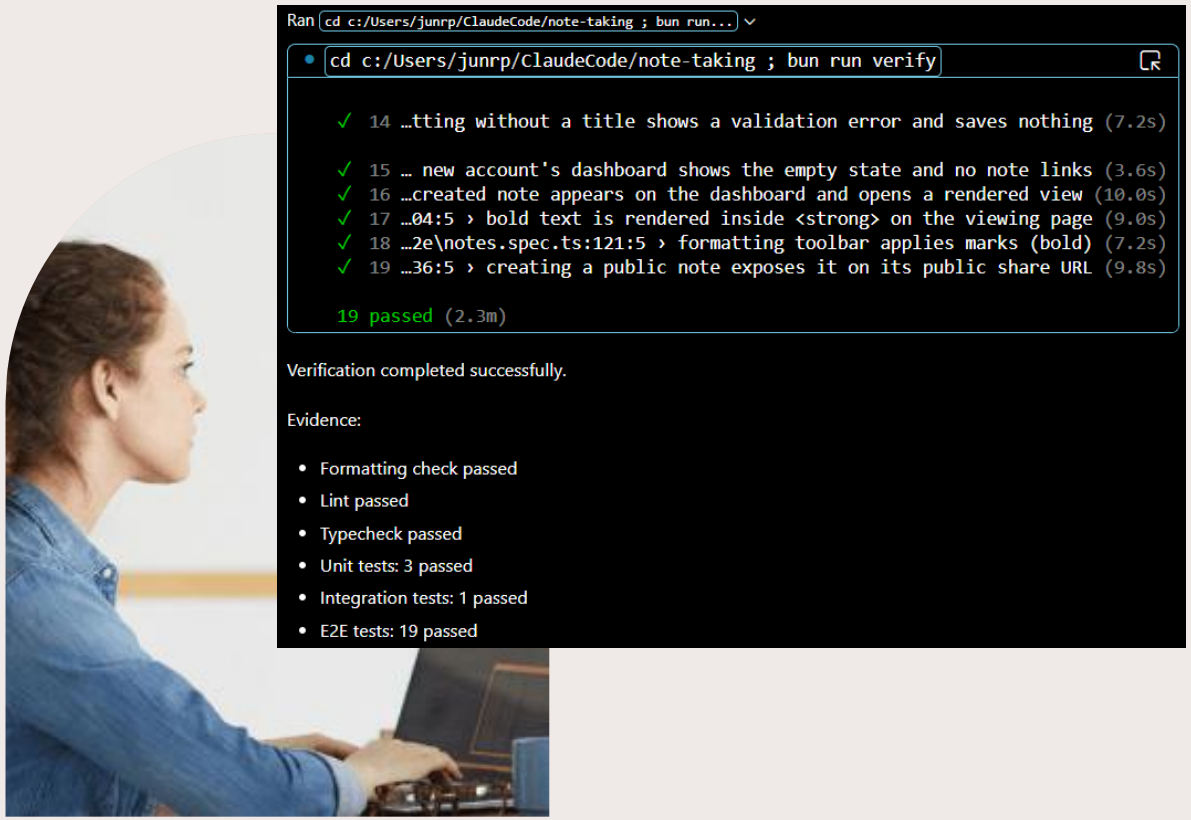
    steps:
      - uses: actions/checkout@v4
      - uses: superfly/flyctl-actions/setup-flyctl@master
      - run: flyctl deploy --remote-only
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup flyctl
        uses: superfly/flyctl-actions/setup-flyctl@master

      - name: Deploy to Fly.io
        run: flyctl deploy --remote-only
        env:
          FLY_API_TOKEN: ${{ secrets.FLY_API_TOKEN }}
```

- Add the **fly-deploy.yml** YAML file shown on the left to the **.github/workflows** folder.
- This file is a GitHub Actions CI/CD configuration file, which provides automated, continuous deployment (CD) for your app.
- It explicitly instructs GitHub's automated servers to perform the following steps every time you push new code to your production branch (usually main):
  - It detects when code is pushed to your primary repository branch.
  - It boots a temporary Linux container on GitHub's infrastructure.
  - It checks out your latest git commit code into that machine.
  - It installs the Fly.io command-line tool (**flyctl**) inside the GitHub environment.
  - It executes **flyctl deploy** using an encrypted authentication token, pushing the changes to your production servers automatically.

TEST PRE-DEPLOY VERIFICATIONS



- Set up persistent **Fly** volume storage for **SQLite** and wire **DATABASE_PATH=/data/notes.db**.
- Add the **FLY_API_TOKEN** GitHub secret and verify the Actions pipeline from a test PR.
- Tighten **fly.toml** for single-machine SQLite behavior if you want to avoid multi-machine write risks.
- Run **bun run verify**.

RUN CI/CD PIPELINE



package.json > { } scripts > abc deploy

```
{
  ▷Debug
  "scripts": {
    "dev": "bun --bun next dev",
    "build": "bun --bun next build",
    "start": "bun --bun next start",
    "format": "oxfmt",
    "format:check": "oxfmt --check",
    "lint": "bunx eslint .",
    "typecheck": "bunx tsc --noEmit",
    "db:migrate": "bunx --bun auth@1.6.23 migrate --yes",
    "db:generate": "bunx --bun auth@1.6.23 generate --yes",
    "test:unit": "bun test tests/unit",
    "test:integration": "bun test tests/integration",
    "test:e2e": "playwright test",
    "test:all": "bun run test:unit && bun run test:integration && bun run test:e2e",
    "verify": "bun run format:check && bun run lint && bun run typecheck && bun run test:all",
    "deploy": "bun run verify && flyctl deploy"
  },
  "dependencies": {
    "@tiptap/pm": "^3.27.3",
```

bun run deploy

4:37 PM

I'm running the deployment command again from the active terminal context with Fly CLI available on PATH.

Ran `cd c:/Users/junrp/ClaudeCode/note-taking && $env:P...`

The previous attempt tripped over PowerShell syntax, so I'm retrying with the correct separator.

Ran `cd c:/Users/junrp/ClaudeCode/note-taking; $env:Pat...`

Read content.txt

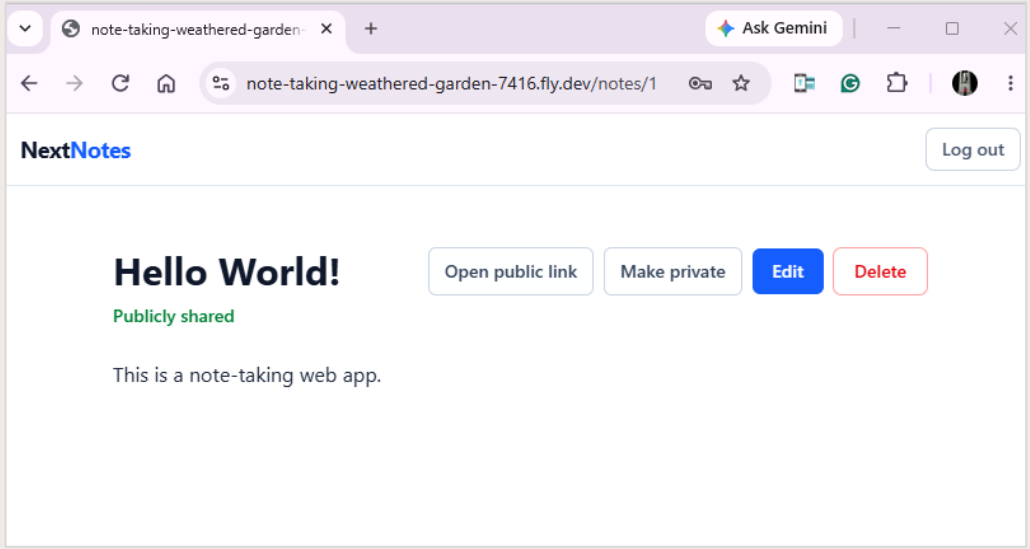
Deployment result

The deployment completed successfully.

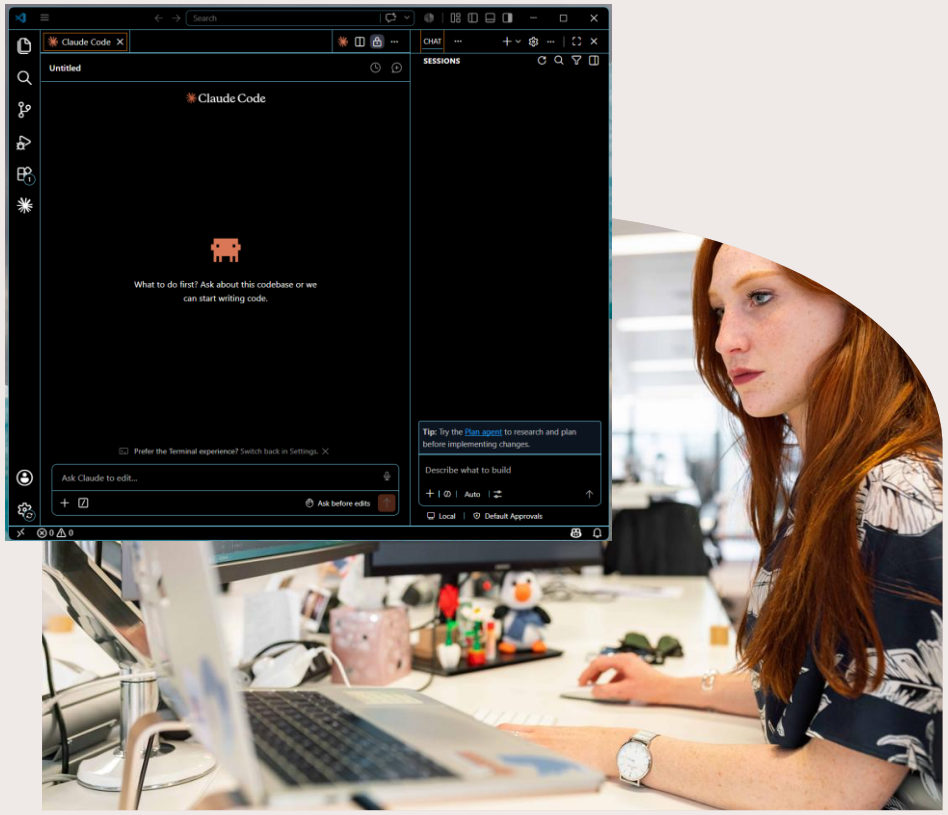
- App URL: https://note-taking-weathered-garden-7416.fly.dev/
- Fly reported the deployment as live and provided the app monitoring URL.

The verification step passed before deployment, and the app is now available.

- Add "deploy": "bun run verify && flyctl deploy" script to package.json so that Claude invokes Fly's CLI tooling via flyctl to carry out deployment.
- Make sure that FLY CLI (flyctl) is installed (at C:/Users/your_name/.fly/bin/flyctl.exe) and authenticated.
- Run bun run deploy.
- The workflow becomes: Deploy request → CI/CD runs: format, lint, type check, unit tests, integration tests, E2E tests, production build → CI/CD passes → Deploy



QUICK REFERENCE



- bun add -d eslint eslint-config-next => Install ESLint tooling as dev dependencies in package.json.
- eslint.config.mjs => ESLint Flat Config file in ECMAScript Modules (ESM) format
- bun run lint => Run linting checks.
- fly.toml => Standard infrastructure configuration file for Fly.io
- fly-deploy.yml => GitHub Actions CI/CD configuration file, which provides automated, continuous deployment (CD) for your app by executing **flyctl deploy**.
- bun run verify => Run all required verifications using the **verify** command defined in **package.json**.
- "deploy": "bun run verify && flyctl deploy" => Command added to **package.json scripts** to invoke Fly's CLI tooling via **flyctl** to carry out deployment
- bun run deploy => Deploy the app using the **deploy** command defined in **package.json**.

THANK YOU

Dr. June Sung Park

june@kosta-online.com

www.kosta-online.com